

Hybrid Collaborative Filtering Implementation Report

Vansh Jain
Roll No: 2023578

MovieLens 100K Dataset

1. Introduction

This report presents a comprehensive implementation of a Hybrid Collaborative Filtering recommendation system on the MovieLens 100K dataset. The system combines user-based and item-based collaborative filtering approaches with adaptive weighting to leverage the strengths of both methods. The implementation includes Pearson correlation similarity with significance weighting, bias-aware predictions, and comprehensive evaluation across five train-test splits.

2. Data Loading and Preprocessing

2.1 Data Loading Function

The `load_data` function handles loading the train and test datasets for each of the five splits (u1-u5) in the MovieLens 100K dataset. Each split contains user ratings with columns: `user_id`, `movie_id`, `rating`, and `timestamp`. The function provides basic statistics about the size of training and test sets.

2.2 Matrix Creation

The `create_matrix` function transforms the tabular rating data into a user-item matrix format, where rows represent users, columns represent movies, and cells contain ratings. Missing values (unrated items) are filled with zeros. This sparse matrix representation is essential for computing similarity metrics and making predictions.

2.3 Statistical Analysis

The `stats` function computes key statistical measures including the overall mean rating, user-specific mean ratings, and item-specific mean ratings. It also calculates user bias and item bias by subtracting the overall mean from respective means. These bias terms are crucial for handling systematic differences in rating patterns and improving prediction accuracy.

3. Similarity Computation

3.1 Pearson Correlation Similarity

The `pearson_similarity` function computes centered cosine similarity (Pearson correlation) between users or items. The implementation follows these steps:

1. Replace zero ratings with NaN to distinguish between 'not rated' and 'rated as zero'
2. Center the ratings by subtracting row or column means
3. Normalize the centered ratings by their magnitudes
4. Compute similarity using matrix multiplication of normalized vectors

3.2 Significance Weighting

To address the reliability issue when similarity is computed from few common ratings, significance weighting is applied. A support matrix counts the number of co-rated items between each pair of users (or movies). The significance factor is computed as $\min(\text{support} / \text{min_support}, 1.0)$, which attenuates similarities based on fewer than `min_support` common ratings. This prevents unreliable similarities from dominating predictions. In this implementation, `min_support` is set to 3.

4. Prediction Methods

4.1 User-Based Collaborative Filtering

The `predict_user` function implements neighborhood-based user collaborative filtering. For a given user-item pair, it identifies `k` most similar users who have rated the target item. The prediction formula is:

$$\text{prediction} = \text{user_mean} + \frac{\sum(\text{similarity} \times (\text{neighbor_rating} - \text{neighbor_mean}))}{\sum(\text{similarity})}$$

Key features include: (1) Handling cold-start by returning mean + bias when no neighbors exist, (2) Selecting top `k_users=40` neighbors by similarity, (3) Filtering to only positive similarities to avoid negative correlations, and (4) Weighted averaging based on neighborhood similarities.

4.2 Item-Based Collaborative Filtering

The `predict_item` function implements item-based collaborative filtering with a symmetric approach to user-based filtering. It finds k most similar items that the user has rated and predicts the rating using:

$$\text{prediction} = \text{item_mean} + \frac{\sum(\text{similarity} \times (\text{user_rating} - \text{neighbor_item_mean}))}{\sum(\text{similarity})}$$

This method is particularly effective when items have many ratings but users have few ratings. The implementation uses `k_items=40` and applies the same principles of bias correction and similarity-weighted averaging.

4.3 Hybrid Approach with Adaptive Weighting

The `predict_hybrid` function combines user-based and item-based predictions using an adaptive weighting scheme. The key innovation is that the weight α adapts based on data availability:

$$\alpha_{\text{adaptive}} = \text{user_rating_count} / (\text{user_rating_count} + \text{item_rating_count})$$

This means when a user has rated many items, the system trusts user-based CF more (α closer to 1). When an item has many ratings but the user has few, it trusts item-based CF more (α closer to 0). The final prediction is: $\text{hybrid_prediction} = \alpha \times \text{user_prediction} + (1 - \alpha) \times \text{item_prediction}$. For this implementation, a base α of 0.5 is used, which balances both approaches equally when adaptive weighting cannot be computed. Predictions are clipped to the valid rating range [1, 5].

5. Evaluation and Experimental Setup

The `evaluate_predictions` function generates predictions for all test cases and computes Normalized Mean Absolute Error (NMAE). NMAE normalizes MAE by the rating range (4.0 for a 1-5 scale), making it interpretable as the average prediction error as a fraction of the scale.

5.1 Hyperparameters

The system was evaluated with the following hyperparameters:

- `k_users` = 40 (number of similar users for user-based CF)
- `k_items` = 40 (number of similar items for item-based CF)
- `min_support` = 3 (significance weighting threshold)
- α = 0.5 (base weighting factor for hybrid approach)

5.2 Evaluation Protocol

The implementation was evaluated using 5-fold cross-validation with the predefined u1-u5 splits from MovieLens 100K. For each split, the model was trained on the base file and tested on the corresponding test file. This ensures robust evaluation across different train-test partitions.

6. Results

The hybrid model demonstrates consistent performance across all splits, with very low standard deviation indicating robustness to different train-test partitions. The achieved NMAE values confirm the effectiveness of combining user-based and item-based collaborative filtering with adaptive weighting.

Split	NMAE
u1	0.18144
u2	0.17933
u3	0.17909
u4	0.17882
u5	0.18005
Average	0.17975
Std Dev	0.00094

The results show an average NMAE of 0.17975 with a standard deviation of only 0.00094, demonstrating exceptional consistency. Split u4 achieved the best performance (0.17882), while split u1 had the highest error (0.18144). The minimal variance across splits indicates that the hybrid approach with adaptive weighting is robust and not overly sensitive to the particular train-test partition.

7. Implementation Architecture

The implementation is organized into five main components:

- 1. Data Loading Module:** Handles reading train/test splits and creating user-item matrices
- 2. Similarity Computation Module:** Computes Pearson correlation with significance weighting for both users and items
- 3. User-Based CF Module:** Implements k-nearest neighbor prediction with bias correction
- 4. Item-Based CF Module:** Implements item similarity-based prediction with bias correction
- 5. Hybrid Prediction Module:** Combines both approaches with adaptive weighting and manages the evaluation pipeline

The workflow follows this sequence: load data → create matrix → compute statistics → compute similarities → generate predictions → evaluate NMAE. This modular design allows easy modification of individual components while maintaining the overall system integrity.

8. Conclusion

This implementation successfully demonstrates the effectiveness of hybrid collaborative filtering on the MovieLens 100K dataset. The adaptive weighting mechanism allows the system to dynamically leverage user-based or item-based CF depending on data availability, resulting in robust and consistent performance. The average NMAE of 0.17975 (approximately 18% error on the rating scale) indicates strong predictive accuracy. Key strengths include comprehensive bias handling, significance-weighted similarities, and the adaptive hybrid approach that combines the best of both collaborative filtering paradigms.

Future enhancements could explore matrix factorization techniques, deep learning approaches, or incorporating content-based features to further improve recommendation quality, particularly for cold-start scenarios.

9. References

- [1] Herlocker, J. L., Konstan, J. A., Borchers, A., & Riedl, J. (1999). An Algorithmic Framework for Performing Collaborative Filtering.
- [2] Papagelis, M., & Plexousakis, D. (2005). Qualitative Analysis of User-Based and Item-Based Prediction Algorithms for Recommendation Agents.
- [3] Harper, F. M., & Konstan, J. A. (2015). The MovieLens Datasets: History and Context. ACM Transactions on Interactive Intelligent Systems.